DR. BABASAHEB AMBEDKAR MARATHWADA UNIVERSITY, CHHATRAPATI SAMBHAJINAGAR

Department of Computer Science & Information Technology

LAB MANUAL

FUNDAMENTALS OF MACHINE LEARNING

USING PYTHON

Course Code : AIT/MJ/501

Programme : M.Sc. Artificial Intelligence

Semester : I

Academic Year : 2026-27

Prepared By

Dr. Manoj Mhaske

Assistant Professor

Department of Computer Science & IT

Laboratory Rules

1. Students must attend practicals regularly.
2. Maintain observation and record books.
3. Programs should be executed individually.
4. Submit assignments within time.
5. Follow laboratory discipline.
6. Switch off the computer after practical completion.
7. Save programs in personal folders.
8. Maintain backup of datasets.

Prerequisites

Students should have basic knowledge of

- Python Programming
- Variables and Data Types
- Loops and Functions
- NumPy
- Pandas
- Matplotlib
- Statistics
- Linear Algebra
- Probability
- Basic Data Visualization

Software Requirements

- Python 3.12+
- Jupyter Notebook
- VS Code / Spyder
- Anaconda

Libraries

- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-Learn
- TensorFlow (optional)
- OpenCV
- NLTK

Hardware Requirements

- Intel i3/i5/i7 Processor
- Minimum 8 GB RAM
- Windows/Linux
- Internet Connection

Course Objectives

After completion of this laboratory, students will be able to

1. Understand machine learning concepts.
2. Implement supervised learning algorithms.
3. Implement unsupervised learning algorithms.
4. Develop predictive models.
5. Evaluate machine learning models.
6. Apply Bayesian learning.
7. Implement clustering algorithms.
8. Develop recommender systems.
9. Perform sentiment analysis.
10. Understand evolutionary algorithms.

Course Outcomes

After successful completion of the course students will be able to

CO1
Understand machine learning workflow and preprocessing.

CO2
Develop regression and classification models.

CO3
Apply Bayesian and probabilistic learning techniques.

CO4
Implement clustering and graphical models.

CO5
Develop intelligent applications using machine learning.

Learning Outcomes

Students will learn

- Data preprocessing
- Feature engineering
- Model training
- Model testing
- Performance evaluation
- Classification
- Regression
- Clustering
- Bayesian learning
- Genetic algorithms
- ML applications

List of Practical Experiments

Practical No.	Title
1	Introduction to Python Libraries for Machine Learning (NumPy, Pandas, Matplotlib, Scikit-Learn)
2	Data Preprocessing using Pandas (Missing Values, Encoding and Normalization)
3	Implementation of Linear Regression
4	Implementation of Multiple Linear Regression
5	Implementation of Logistic Regression for Classification
6	Implementation of Decision Tree
7	Implementation of K-Nearest Neighbors (KNN) Classifier
8	Implementation of Support Vector Machine
9	Implementation of Naïve Bayes Classifier
10	Implementation of K-Means Clustering
11	Implementation of Artificial Neural Network
12	Implementation of Hidden Markov Model / Markov Chain Simulation
13	Implementation of Genetic Algorithm using Python
14	Sentiment Analysis using Machine Learning
15	Recommendation System using Collaborative Filtering

Practical 1

Title

Introduction to Python Libraries for Machine Learning

Objective

To study basic Python libraries used in Machine Learning.

Theory

Machine Learning relies on scientific computing libraries.

NumPy
Provides multidimensional arrays.

Pandas
Used for data manipulation.

Matplotlib
Creates graphs.

Scikit-learn
Provides ML algorithms.

Algorithm

1. Import required libraries.
2. Create NumPy array.
3. Create Pandas DataFrame.
4. Display information.
5. Plot graph.

Python Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

a=np.array([10,20,30,40,50])

print(a)

df=pd.DataFrame({
    "Student":["A","B","C","D"],
    "Marks":[70,80,90,85]
})

print(df)

plt.plot(df["Marks"])
plt.title("Marks")
plt.show()
```

Expected Output

```
[10 20 30 40 50]

  Student Marks
0 A      70
1 B      80
2 C      90
3 D      85
```

Conclusion

Python scientific libraries simplify machine learning implementation.

Practical No. 2

Title

Data Preprocessing using Pandas (Missing Values, Encoding and Normalization)

Objective

To learn data preprocessing techniques such as handling missing values, encoding categorical variables, and normalizing numerical features using Python.

Theory

Data preprocessing is the process of preparing raw data before applying machine learning algorithms. Since real-world datasets often contain missing values, inconsistent data, and categorical attributes, preprocessing improves data quality and model accuracy.

The major preprocessing techniques are:

- Handling Missing Values
- Encoding Categorical Variables
- Feature Scaling (Normalization/Standardization)

Python libraries used:

- Pandas
- NumPy
- Scikit-learn

Algorithm

1. Import required libraries.
2. Create or load a dataset.
3. Display missing values.
4. Replace missing values using mean().
5. Encode categorical values.
6. Normalize numerical columns.
7. Display the processed dataset.

Python Program

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder,
MinMaxScaler

data = {
    'Age':[23,25,np.nan,30,28],
    'Salary':[25000,30000,28000,np.nan,45000],
    'Gender':['Male','Female','Female','Male','Female']
}

df = pd.DataFrame(data)

print("Original Dataset")
print(df)

# Fill Missing Values
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Salary'].fillna(df['Salary'].mean(), inplace=True)

# Encoding
encoder = LabelEncoder()
df['Gender'] = encoder.fit_transform(df['Gender'])

# Normalization
scaler = MinMaxScaler()
df[['Age','Salary']] =
scaler.fit_transform(df[['Age','Salary']])

print("\nProcessed Dataset")
print(df)
```

Sample Output

```
Original Dataset

    Age   Salary Gender
0 23.0 25000 Male
1 25.0 30000 Female
2 NaN 28000 Female
3 30.0 NaN Male
4 28.0 45000 Female
```

```
Processed Dataset

    Age  Salary  Gender
0 0.00 0.00 1
1 0.29 0.25 0
2 0.47 0.15 0
3 1.00 0.43 1
4 0.71 1.00 0
```

Conclusion

Thus, missing values were handled, categorical values were encoded, and numerical attributes were normalized successfully using Pandas and Scikit-learn.

Practical No. 3

Title

Implementation of Linear Regression

Objective

To implement Linear Regression for predicting continuous values.

Theory

Linear Regression is a supervised learning algorithm used to predict a continuous dependent variable using one independent variable.

Equation:

$Y=mX+c$

where

- X = Independent Variable
- Y = Dependent Variable
- m = Slope
- c = Intercept

Applications:

- House Price Prediction
- Salary Prediction
- Temperature Forecasting

Algorithm

1. Import libraries.
2. Create dataset.
3. Split dataset.
4. Train Linear Regression model.
5. Predict test values.
6. Plot regression line.
7. Display accuracy.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5]).reshape(-1,1)
Y = np.array([2,4,6,8,10])

model = LinearRegression()

model.fit(X,Y)

prediction = model.predict(X)

print("Slope:",model.coef_)
print("Intercept:",model.intercept_)

plt.scatter(X,Y,color='red')
plt.plot(X,prediction)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression")
plt.show()
```

Sample Output

```
Slope : [2.]

Intercept : 0.0
```

Graph:

- Red points represent actual values.
- Straight line represents predicted values.

Conclusion

Thus, Linear Regression was successfully implemented to predict continuous numerical values.

Practical No. 4

Title

Implementation of Multiple Linear Regression

Objective

To implement Multiple Linear Regression using multiple independent variables.

Theory

Multiple Linear Regression predicts a dependent variable using two or more independent variables.

Equation

$Y=a+b1X1+b2X2+\cdots+bnXn$

Applications

- Sales Prediction
- Stock Price Prediction
- Student Performance Prediction

Algorithm

1. Import required libraries.
2. Create dataset with multiple features.
3. Train regression model.
4. Predict output.
5. Display coefficients.

Python Program

```
import pandas as pd
from sklearn.linear_model import LinearRegression

data = {
'Experience':[1,2,3,4,5],
```

```
'Education':[12,14,15,16,18],
'Salary':[25000,32000,40000,48000,60000]
}

df = pd.DataFrame(data)

X = df[['Experience','Education']]
Y = df['Salary']

model = LinearRegression()

model.fit(X,Y)

print("Coefficients")
print(model.coef_)

print("Intercept")
print(model.intercept_)

prediction = model.predict([[6,18]])

print("Predicted Salary =",prediction)
```

Sample Output

```
Coefficients

[6500.25 1800.60]

Intercept

-42000.35

Predicted Salary

[68500.48]
```

Conclusion

Thus, Multiple Linear Regression successfully predicted salary using multiple input variables.

Practical No. 5

Title

Implementation of Logistic Regression for Classification

Objective

To classify data using Logistic Regression.

Theory

Logistic Regression is a supervised classification algorithm that predicts categorical outcomes.

It uses the Sigmoid Function:

P=1/1+e−x

Applications:

- Disease Prediction
- Email Spam Detection
- Customer Churn Prediction
- Loan Approval

Algorithm

1. Import libraries.
2. Load dataset.
3. Split data.
4. Train Logistic Regression model.
5. Predict classes.
6. Display accuracy.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
Y = iris.target

X_train,X_test,Y_train,Y_test = train_test_split(
X,Y,test_size=0.3,random_state=1)

model = LogisticRegression(max_iter=200)

model.fit(X_train,Y_train)

prediction = model.predict(X_test)

print("Predicted Classes")

print(prediction)

accuracy = accuracy_score(Y_test,prediction)

print("Accuracy =",accuracy)
```

Sample Output

```
Predicted Classes

[0 1 1 2 2 0 1 2 0 1 2 ...]

Accuracy = 0.9778
```

Conclusion

Thus, Logistic Regression was successfully implemented to classify data into different categories with high accuracy.